From Model to Medium

Topic 2: API Design Principles

ML + UI + API + MCP Project

Learning Path

October 12, 2026

Outline

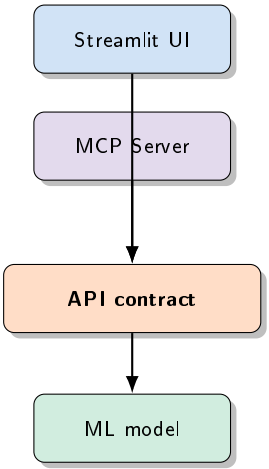
1. Why design matters
2. Endpoints and URLs
3. Schemas and Errors
4. Scaling the API
5. Checklist
6. Takeaways

An API is a contract

Key idea

A well-designed API is predictable: a developer (or an AI assistant) can guess how it works before reading the docs.

- Clear endpoint names
- Consistent request and response shapes
- Helpful, uniform errors
- Stable over time (versioning)



Anatomy of a URL

`https://api.bank.com/v1/models/7/predict?round=2`



Rule

The path identifies the **resource**; query parameters **filter or tweak** the result.

Resource naming: nouns, not verbs

Avoid (verbs)

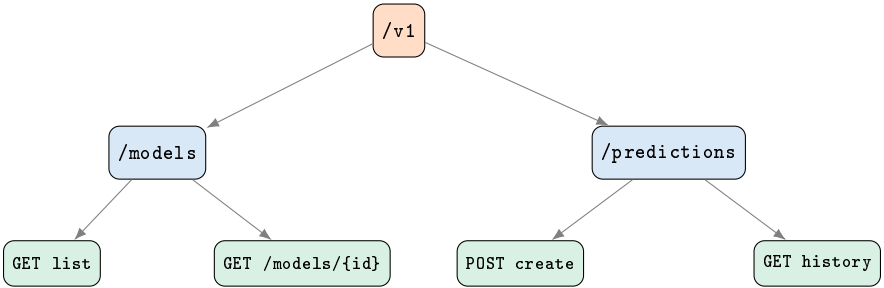
```
/getAllModels
/createPrediction
/deleteModel?id=7
```

Prefer (nouns + method)

```
GET /models
POST /predictions
DELETE /models/7
```

- Use **plural** nouns and `lowercase-with-hyphens`.
- The HTTP method already says the action.
- Nest only when ownership is clear: `/models/7/predictions`.

Resource tree of our ML API



Easy sample: endpoints in FastAPI

```python
from fastapi import FastAPI

app = FastAPI(title="House Price API")

@app.get("/v1/models")                    # list resources
def list_models():
    return [{"id": 1, "name": "linear_regression"}]

@app.get("/v1/models/{model_id}")         # one resource
def get_model(model_id: int):
    return {"id": model_id, "name": "linear_regression"}

@app.post("/v1/predictions")              # create a prediction
def create_prediction(features: dict):
    return {"predicted_price": 412500.0}
```

Consistent request/response schemas

```python
from pydantic import BaseModel

class PredictionRequest(BaseModel):
    square_feet: float
    bedrooms: int

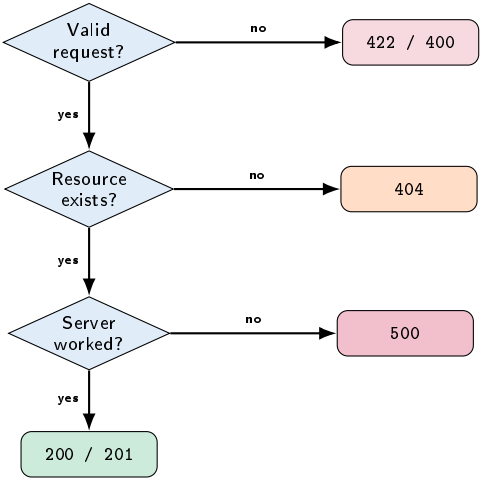
class PredictionResponse(BaseModel):
    predicted_price: float
    model_version: str
```

```json
{
  "predicted_price": 412500.0,
  "model_version": "1.0.0"
}
```

Tip

Same field names and types everywhere: `snake_case`, numbers as numbers.

Choosing the right status code



Predictable error handling

```python
from fastapi import FastAPI, HTTPException

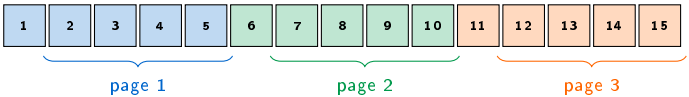
app = FastAPI()
MODELS = {1: "linear_regression"}

@app.get("/v1/models/{model_id}")
def get_model(model_id: int):
    if model_id not in MODELS:
        raise HTTPException(
            status_code=404,
            detail={"code": "MODEL_NOT_FOUND",
                    "message": f"Model {model_id} does not exist"},
        )
    return {"id": model_id, "name": MODELS[model_id]}
```

Rule

Every error uses the same JSON shape: a machine-readable code plus a human message.

Pagination: do not return everything



Request

```
GET /v1/predictions?page=2&limit=5
```

Response envelope

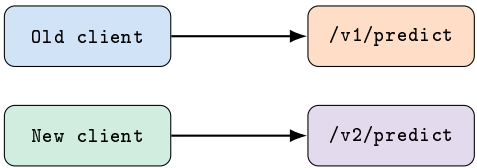
```
{
  "items": [ ... 5 records ... ],
  "page": 2, "limit": 5, "total": 15
}
```

Easy sample: pagination in FastAPI

```python
HISTORY = [{"id": i, "price": 300000 + i * 1000} for i in range(1, 16)]

@app.get("/v1/predictions")
def list_predictions(page: int = 1, limit: int = 5):
    start = (page - 1) * limit
    items = HISTORY[start : start + limit]
    return {"items": items, "page": page,
            "limit": limit, "total": len(HISTORY)}
```

Versioning: change without breaking clients



- Put the version in the path: `/v1`, `/v2`.
- Adding optional fields is safe; renaming or removing is breaking.
- Keep `v1` alive while clients migrate.

Good vs. poor API design

Aspect	Poor	Good
Naming	`/getModelData`	`GET /models/7`
Errors	Plain text, random codes	Uniform JSON + status code
Lists	Returns all rows	Paginated with `total`
Changes	Edits live endpoint	New `/v2` version
Fields	`SqFt`, `bed_Rooms`	Consistent `snake_case`

Takeaways

1. An API is a contract: keep it **predictable and consistent**.
2. Name resources with **plural nouns**; let the HTTP method be the verb.
3. Define schemas (Pydantic) so inputs and outputs never surprise anyone.
4. Return the **right status code** with one standard error shape.
5. **Paginate** lists and **version** the API to avoid breaking clients.
6. Next topic: **FastAPI (Python Web Framework)**.